Growing Code

JavaScript Fundamentals Through Garden Data

Summary: Discover JavaScript's fundamental syntax and semantics by analyzing community garden data. Learn expressions, variables, functions, and control flow.

Version: 3.0

Contents

1 Foreword

In community gardens across the world, data tells stories of growth, collaboration, and impact. From tracking harvest yields that feed local families to monitoring water usage that preserves precious resources, every measurement matters. Each data point represents someone's dedication—a volunteer's weekend hours, a child's first tomato, an elderly neighbor sharing decades of gardening wisdom.

JavaScript, much like these gardens, has grown from simple scripting roots into an essential ecosystem powering interactive applications, automated pipelines, and server-side runtime environments like Node.js. Today, JavaScript enables communities to optimize resource sharing and empowers developers to transform raw data into meaningful insights.

In this project, you'll discover JavaScript's fundamental building blocks while analyzing real community garden data. You'll learn that programming, like gardening, is about nurturing growth—both in code and in the communities we serve.

2 AI Instructions

2.1 Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

2.2 Main Message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills—both coding and non-coding—that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Good & Bad Practices

Good practice: I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

Bad practice: I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility and I fail my project.

3 Introduction

Welcome to **Growing Code**!

In this project, you'll discover JavaScript's core concepts through community garden scenarios. You'll work with practical exercises that introduce fundamental programming concepts in an engaging, real-world context.

IMPORTANT

For each exercise, you must write **ONLY** a function (not a main program). Each file should contain only the requested function that handles input and output directly. Do not call the function directly in your file, and do not write code outside functions. Export your functions using standard CommonJS module exports: `module.exports = ft_function_name;`.

4 General Instructions

- Your functions must be written in **Node.js (v18+)**.
- Your code must adhere to standard ESLint rules (no global leakages, 2-space indentation).
- Each exercise must be in its own file.
- Each file should contain only the requested function.
- Function names must match exactly what is requested.
- For negative numbers or invalid inputs, the behaviour is undefined.
- Type hints via JSDoc comments are optional for exercises 0 to 6, but required for Exercise 7.

Input Note for JavaScript

Standard Node.js stdin execution is asynchronous. To keep the execution synchronous matching the original specification, you are authorized to use `readline-sync` (`npm install readline-sync`) for user input prompts.

5 Exercise 0: Hello Garden

- **Directory:** `ex0/`
- **Files to Submit:** `ft_hello_garden.js`
- **Authorized:** `console.log()`

Write a function named `ft_hello_garden` that displays a welcome message to the garden community.

```
ft_hello_garden();
// Output:
// Hello, Garden Community!
```

6 Exercise 1: Garden Name

- **Directory:** `ex1/`
- **Files to Submit:** `ft_garden_name.js`
- **Authorized:** `readline-sync.question()`, `console.log()`

Write a function named `ft_garden_name` that asks for a garden name, then displays it followed by a fixed status message.

```
Enter garden name: Community Garden
Garden: Community Garden
Status: Growing well!
```

7 Exercise 2: Garden Plot Area

- **Directory:** `ex2/`
- **Files to Submit:** `ft_plot_area.js`
- **Authorized:** `readline-sync.question()`, `parseInt()`, `console.log()`

Write a function named `ft_plot_area` that asks for length and width, then calculates and displays the area.

```
Enter length: 5
Enter width: 3
Plot area: 15
```

8 Exercise 3: Harvest Total

- **Directory:** `ex3/`
- **Files to Submit:** `ft_harvest_total.js`
- **Authorized:** `readline-sync.question()`, `parseInt()`, `console.log()`

Write a function named `ft_harvest_total` that asks for the weight of harvests across three days and calculates the total.

```
Day 1 harvest: 5
Day 2 harvest: 8
Day 3 harvest: 3
Total harvest: 16
```

9 Exercise 4: Plant Age Check

- **Directory:** `ex4/`
- **Files to Submit:** `ft_plant_age.js`
- **Authorized:** `readline-sync.question()`, `parseInt()`, `console.log()`

Write a function named `ft_plant_age` that asks for a plant's age in days and checks if it's ready to harvest (> 60 days).

```
Enter plant age in days: 75
Plant is ready to harvest!
```

10 Exercise 5: Water Reminder

- **Directory:** `ex5/`
- **Files to Submit:** `ft_water_reminder.js`
- **Authorized:** `readline-sync.question()`, `parseInt()`, `console.log()`

Write a function named `ft_water_reminder` that asks for days since the last watering. Print `Water the plants!` if > 2 days, else `Plants are fine`.

11 Exercise 6: Count to Harvest

- **Directory:** `ex6/`
- **Files to Submit:** `ft_count_harvest_iterative.js`, `ft_count_harvest_recursive.js`
- **Authorized:** `readline-sync.question()`, `parseInt()`, `console.log()`, helper functions

Write iterative and recursive implementations counting from 1 to a specified number of harvest days.

```
Days until harvest: 5
Day 1
Day 2
Day 3
Day 4
Day 5
Harvest time!
```

12 Exercise 7: Seed Inventory with Type Annotations

- **Directory:** `ex7/`
- **Files to Submit:** `ft_seed_inventory.js`
- **Authorized:** `console.log()`, String methods

Write `ft_seed_inventory` to display seed information. You must document the function using JSDoc type annotations:

```
/**
 * @param {string} seedType
 * @param {number} quantity
 * @param {string} unit
 * @returns {void}
 */
function ft_seed_inventory(seedType, quantity, unit) {
  // Implementation
}
```

- `"packets"` → `Tomato seeds: 15 packets available.`
- `"grams"` → `Carrot seeds: 8 grams total`
- `"area"` → `Lettuce seeds: covers 12 square meters`
- Any other unit → `Unknown unit type`

13 Turn in and Submission

Turn in your assignment to your Git repository as usual. Only work inside your repository will be evaluated during defense. Ensure all function exports and file naming conventions match the prompt requirements.